1. Estrutura da rede neural

A rede neural artificial é do tipo feedforward e possui três camadas: uma camada de entrada, uma camada oculta e uma camada de saída. Os parâmetros definidos no código determinam a quantidade de exemplos e de neurônios:

Parâmetro	Valor	Significado
N	64	Exemplos de treinamento
D_in	1000	Características de entrada por exemplo
H	100	Neurônios da camada oculta
D_out	10	Valores produzidos na saída

A arquitetura é, portanto, 1000 → 100 → 10. Os 64 exemplos são processados em conjunto a cada iteração.

2. Dados e pesos

```
x, y = randn(N, D_in), randn(N, D_out)
w1, w2 = randn(D_in, H), randn(H, D_out)
```

- x: matriz de entradas com dimensão 64 × 1000.
- y: matriz de saídas desejadas com dimensão 64 × 10.
- w1: pesos entre a entrada e a camada oculta, dimensão 1000 × 100.
- w2: pesos entre a camada oculta e a camada de saída, dimensão 100 × 10.

A função randn gera valores aleatórios de uma distribuição normal padrão. Não há dados de um problema real nesse exemplo: as entradas, os alvos e os pesos iniciais são gerados aleatoriamente.

3. Forward pass: propagação para frente

```
h = 1 / (1 + np.exp(-x.dot(w1)))
y_pred = h.dot(w2)
loss = np.square(y_pred - y).sum()
print(t, loss)
```

Primeiro, o produto matricial x.dot(w1) combina entradas e pesos. Sobre o resultado, aplica-se a função sigmoide, obtendo a matriz h de ativações da camada oculta (64 × 100). Depois, h.dot(w2) fornece y_pred, a previsão da rede (64 × 10). Por fim, o programa mede a diferença entre previsão e alvo por meio de loss e mostra o erro na tela.

4. Funções matemáticas principais

4.1 Função de ativação sigmoide

A sigmoide é dada por σ(z) = 1 / (1 + e^(−z)). Ela introduz não linearidade e produz valores entre 0 e 1. Neste código, é aplicada somente à camada oculta; a camada de saída é linear.

4.2 Função de perda

A função de perda utilizada é $L = \Sigma(y_pred - y)^2$, ou seja, a soma dos quadrados das diferenças entre previsões e saídas desejadas. O objetivo do treinamento é diminuir essa quantidade. O código utiliza a soma, não a média dos erros.

5. Backpropagation: cálculo dos gradientes

```
grad_y_pred = 2.0 * (y_pred - y)
grad_w2 = h.T.dot(grad_y_pred)
grad_h = grad_y_pred.dot(w2.T)
grad_w1 = x.T.dot(grad_h * h * (1 - h))
```

- grad_y_pred: derivada da perda em relação à previsão.
- grad_w2: gradiente dos pesos que conectam a camada oculta à saída.
- grad_h: gradiente propagado da saída para as ativações ocultas.
- grad_w1: gradiente dos pesos que conectam a entrada à camada oculta.

No cálculo de grad_w1, o termo h * (1 − h) corresponde à derivada da sigmoide. A operação de retropropagação usa a regra da cadeia para descobrir como cada peso influencia o erro.

6. Atualização dos pesos: descida do gradiente

```
w1 -= 1e-4 * grad_w1
w2 -= 1e-4 * grad_w2
```

As duas instruções aplicam a regra w_novo = w_antigo − η · gradiente. A taxa de aprendizagem η = 1e−4 = 0,0001 controla o tamanho de cada ajuste. Subtrair o gradiente é uma maneira de procurar valores de pesos que reduzam a perda.

7. Ciclo de treinamento

1. Definir as dimensões da rede e inicializar os dados e pesos.
2. Calcular a camada oculta e a saída prevista (forward pass).
3. Calcular o erro total (loss).
4. Calcular os gradientes por retropropagação.
5. Atualizar w1 e w2 por descida do gradiente.
6. Repetir o procedimento por 2000 iterações.

O laço for t in range(2000) executa as iterações t = 0, 1, ..., 1999. A função de perda é exibida antes da atualização dos pesos de cada iteração.

8. Conclusão

O código demonstra o treinamento de uma rede neural simples usando apenas NumPy. A rede calcula previsões, mede o erro, encontra os gradientes e atualiza seus pesos repetidamente. Esse processo permite entender os fundamentos do aprendizado em redes neurais.